Package 'stratsampling'

January 15, 2026

Title Implementation of the stratified sampling for coding

Version 0.0.0.9000

Description This code implements the methods described in the stratified coding paper of Mozer, Pashley and Miratrix. It also has the code to run the simulations for that paper.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends tidyverse

RoxygenNote 7.3.3

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Contents

calc_optimal_allocation_oracle . . . 2
calc_sampling_weights . . . 2
compare_stratifications . . . 3
compare_stratifications_precoding . . . 3
compute_variance_decomposition . . . 4
create_stratification_candidates . . . 4
est_ma_srs . . . 6
est_ma_srs_old . . . 6
est_ma_strat . . . 7
est_oracle . . . 8
est_subset . . . 8
evaluate_stratification . . . 9
evaluate_stratification_precoding . . . 10
generate_observed_sample . . . 11
generate_population . . . 11
generate_sample_index . . . 12
plot_strata_residuals . . . 13

Index **14**

`calc_optimal_allocation_oracle` *This method assumes we have Yobs known for everyone, and thus is an oracle. It will look at the actual residuals (Yobs - Yhat) to determine optimal allocation of samples across strata (and treatment arms if Z passed) to minimize variance of the model-assisted estimator.*

Description

This method assumes we have Yobs known for everyone, and thus is an oracle. It will look at the actual residuals (Yobs - Yhat) to determine optimal allocation of samples across strata (and treatment arms if Z passed) to minimize variance of the model-assisted estimator.

Usage

```
calc_optimal_allocation_oracle(
  dat,
  n_code,
  Z = NULL,
  strata = NULL,
  Yobs = "Yobs",
  Yhat = "Yhat",
  return_counts = FALSE,
  seed = NULL
)
```

`calc_sampling_weights` *Given a sampling indicator and stratification, calculate the associated sampling weights.*

Description

Given a sampling indicator and stratification, calculate the associated sampling weights.

Usage

```
calc_sampling_weights(dat, Z = NULL, S = "S", strata = NULL)
```

Arguments

`S`	Name of column in dat that is the sampling indicator (0/1). Or vector of values.

Value

Vector of sampling probabilities (weights) for each unit in dat.

compare_stratifications

Compare Multiple Candidate Stratification Variables

Description

Evaluates and compares multiple candidate stratification variables to help select the best one for stratified model-assisted estimation. Each candidate is assessed using `evaluate_stratification`, and results are summarized in a ranked table.

Usage

```
compare_stratifications(data, candidates, Z = "Z", ...)
```

Arguments

`data`	Data frame containing a `resid` column (residuals = Yobs - Yhat), a treatment indicator column named `"Z"`, and any columns referenced by the candidate stratifications.
`candidates`	Named list of candidate stratification vectors. Each element should be a vector of the same length as `nrow(data)` containing stratum assignments. Names are used to identify stratifications in the output.
`min_var_threshold`	Numeric. Minimum within-stratum residual variance required. Stratifications with any stratum below this threshold are excluded from results. Default is `0.01`.

Value

A list of performance metrics for each stratification method

compare_stratifications_precoding

Compare candidate stratifications using pre-coding metrics

Description

Compare candidate stratifications using pre-coding metrics

Usage

```
compare_stratifications_precoding(data, candidates, Z = NULL)
```

Arguments

`data`	Data frame with Yhat and Z columns
`candidates`	Named list of stratification vectors
`Z`	Character string specifying the name of the treatment indicator or `NULL` to ignore treatment groups.

Value

Data frame comparing stratifications

`compute_variance_decomposition` *Compute variance decomposition (BS and WS components from Theorem 3.2)*

Description

Compute variance decomposition (BS and WS components from Theorem 3.2)

Usage

```
compute_variance_decomposition(dat, Z = "Z", strata = NULL)
```

Arguments

`dat`	Data frame with Yobs, Yhat
`Z`	Treatment indicator (if applicable)
`strata`	Stratification variable

Value

Data frame with BS and WS components by treatment arm (if applicable) and overall

---

`create_stratification_candidates` *Create Candidate Stratification Variables*

Description

Generates a set of candidate stratification variables for use in stratified model-assisted estimation. Creates single-variable stratifications (tertiles, quartiles, quintiles) for each specified variable, as well as interaction stratifications for all pairs of variables.

Usage

```
create_stratification_candidates(
  dat,
  vars,
  Z = NULL,
  jitter_vars = NULL,
  sd_jitter = 0.001,
  seed = NULL
)
```

Arguments

dat	Data frame containing the variables to stratify on.
vars	Character vector specifying the names of columns in dat to use for creating stratifications.
Z	Optional character string specifying the name of the treatment indicator variable in dat. If provided, stratifications are computed separately within each treatment arm, ensuring balanced strata across arms. Default is NULL (pooled stratification).
jitter_vars	Character vector specifying which variables (subset of vars) should have small random noise added before computing quantile breaks. This helps avoid ties that can cause cut() to fail. Default is NULL (no jitter applied).
sd_jitter	Numeric value specifying the standard deviation of the jitter noise. Default is 1e-3.
seed	Optional integer seed for reproducibility when jitter is applied. Default is NULL.

Details

For each variable in vars, the function creates three stratifications based on quantiles:

- Tertiles (3 groups): Low, Med, High
- Quartiles (4 groups): Low, Med-Low, Med-High, High
- Quintiles (5 groups): Very Low, Low, Med, High, Very High

For each unique pair of variables, the function creates four interaction stratifications by crossing quantile groups:

- 2x2: median split on both variables (4 strata)
- 2x3: median split on first, tertiles on second (6 strata)
- 3x2: tertiles on first, median split on second (6 strata)
- 3x3: tertiles on both variables (9 strata)

When Z is provided, quantile breaks are computed separately within each treatment arm. This ensures that the same proportion of treated and control units fall into each stratum level, which is important for maintaining balance in stratified estimation.

When a variable has many tied values, quantile breaks may not be unique, causing cut() to fail. Use jitter_vars to add small random noise (SD = 0.001) to such variables before computing breaks.

Value

A named list of factor vectors, each representing a candidate stratification. Names follow the convention:

Single-variable {var}_tertiles, {var}_quartiles, {var}_quintiles

Interactions {var1}_{var2}_2x2, {var1}_{var2}_2x3, {var1}_{var2}_3x2, {var1}_{var2}_3x3

Factor levels have meaningful labels (e.g., "Low Yhat", "Med Yhat / High word_count"). Stratifications that fail due to non-unique quantile breaks are automatically excluded from the output.

`est_ma_srs` *Calculate treatment impacts with SRS sample*

Description

(Baseline) function to calculate tau_hat_ma (the model-assisted treatment effect estimator) assuming we have human-coded scores, Yobs, for a subsample of n<N documents plus predicted scores for all N documents, with the subsample chosen using SRS

Usage

```
est_ma_srs(
  data,
  Yhat = "Yhat",
  Yobs = "Yobs",
  S = "S",
  Z = "Z",
  finite_pop = FALSE
)
```

`est_ma_srs_old` *(Baseline) function to calculate tau_hat_ma (the model-assisted treatment effect estimator) assuming we have human-coded scores, Yobs, for a subsample of n<N documents plus predicted scores for all N documents, with the subsample chosen using SRS*

Description

(Baseline) function to calculate tau_hat_ma (the model-assisted treatment effect estimator) assuming we have human-coded scores, Yobs, for a subsample of n<N documents plus predicted scores for all N documents, with the subsample chosen using SRS

Usage

```
est_ma_srs_old(
  data,
  Yhat = "Yhat",
  Yobs = "Yobs",
  S = "S",
  Z = "Z",
  return_pieces = FALSE
)
```

`est_ma_strat` *Calculate treatment impacts with stratified sample*

Description

Function to calculate the model-assisted treatment effect estimator assuming we have K fixed strata and hand-coded scores for a subsample of n<N documents chosen by stratified random sampling (with fixed and known sampling probabilities).

Usage

```
est_ma_strat(
  data = NULL,
  Yhat = "Yhat",
  Yobs = "Yobs",
  S = "S",
  Z = "Z",
  strata = "strata",
  return_pieces = FALSE,
  finite_pop = FALSE
)
```

Details

If strata set to null, then do the SRS version.

Value

A data.frame with columns:

`est`	the point estimate
`SE`	Not sure what this is.
`SEhat`	the estimated standard error of the estimator
`SEhat.full`	the estimated standard error component due to full coding variance (i.e., if we had coded everything)
`SEhat.samp`	the estimated standard error component due to sampling variance (i.e., due to only coding a subset of documents) HOW DIFFERENT FROM SEhat?
`S2.0`	the estimated population variance of the outcomes in the control group
`S2.1`	the estimated population variance of the outcomes in the treatment group

est_oracle *Compute oracle estimate (full coding) using shared functions*

Description

Compute oracle estimate (full coding) using shared functions

Usage

```
est_oracle(data, Yobs = "Yobs", Z = "Z")
```

Value

Data frame with oracle estimate and SE

est_subset *Simple subset coding estimator*

Description

This is the (Baseline) function to calculate tau_hat_subset (the simple subset treatment effect estimator) assuming we have human-coded scores, Yobs, for a subsample of n<N chosen using SRS

Usage

```
est_subset(data, Yobs = "Yobs", S = "S", Z = "Z", finite_pop = FALSE)
```

Value

A data.frame with columns:

est	the point estimate
SE	The SE of the estimator if everything were coded
SEhat	the estimated standard error of the estimator
SEhat.full	NA (not applicable for this estimator)
SEhat.samp	NA (not applicable for this estimator)
S2.0	the sample variance of the coded outcomes in the control group
S2.1	the sample variance of the coded outcomes in the treatment group

evaluate_stratification *Evaluate a Candidate Stratification Variable*

Description

Computes stratum-level residual statistics and heterogeneity metrics to assess the potential benefit of a stratification for model-assisted estimation.

Usage

```
evaluate_stratification(data, strata_var, Z = "Z", min_var_threshold = 0.01)
```

Arguments

data	Data frame containing a resid column (residuals = Yobs - Yhat) and the stratification variable specified by strata.
strata_var	Character string specifying the name of the column to use as the stratification variable.
Z	Character string specifying the name of the treatment indicator or NULL to ignore treatment groups.
min_var_threshold	Numeric. Minimum within-stratum residual variance required. If any stratum has variance below this threshold, the function returns NULL. This guards against strata with near-zero variance that could cause numerical issues. Default is 0.01.

Details

This function is useful for comparing candidate stratification variables before selecting one for stratified sampling. Key metrics to examine:

- var_mean_resid: Higher values indicate greater between-strata variance (BS), suggesting stratification will reduce sampling variance.
- ratio_var: Values greater than 2 indicate heteroskedasticity across strata, suggesting optimal (Neyman) allocation will outperform proportional allocation.
- imbalance: Values close to 1 indicate balanced strata; highly unbalanced strata may reduce efficiency.

Value

A list with three elements, or NULL if the stratification is invalid (e.g., empty data, missing values, or variance below threshold):

strata_stats Data frame with stratum-level statistics:

- N_zk (or N_k): Sample size in each stratum
- mean_resid: Mean residual in each stratum
- var_resid: Variance of residuals in each stratum
- sd_resid: Standard deviation of residuals in each stratum

If Z is specified, includes treatment indicator Z.

`heterogeneity` Data frame summarizing heterogeneity across strata:

- `n_strata`: Number of strata
- `min_size`, `max_size`: Range of stratum sizes
- `imbalance`: Ratio of largest to smallest stratum size
- `min_var`, `max_var`: Range of residual variances
- `ratio_var`: Ratio of max to min residual variance (heteroskedasticity measure)
- `range_var`: Difference between max and min residual variance
- `var_mean_resid`: Variance of stratum mean residuals (proxy for BS)

If `Z` is specified, returns one row per treatment arm.

`stratification` Character string echoing the stratification variable name.

`evaluate_stratification_precoding` *Evaluate a candidate stratification strategy using pre-coding proxies*

Description

Since we don't have residuals, we use proxies:

- Variance of Yhat within strata (lower = more homogeneous predictions)
- Spread of stratum means (higher = better potential for bias correction)
- Balance of stratum sizes

Usage

```
evaluate_stratification_precoding(data, strata = NULL, Z = "Z")
```

Arguments

`data`	Data frame with Yhat column
`strata`	Stratification vector
`Z`	Character string specifying the name of the treatment indicator or `NULL` to ignore treatment groups.

Value

Data frame with proxy quality metrics

`generate_observed_sample` *Given a fully observed population, sample and generate observed outcomes, given possibly stratification, etc. This method would be used for simulation.*

Description

Given a fully observed population, sample and generate observed outcomes, given possibly stratification, etc. This method would be used for simulation.

Usage

```
generate_observed_sample(
  dat,
  n_code,
  Z = NULL,
  strata = NULL,
  by_tx = FALSE,
  seed = NULL
)
```

Arguments

n_code	Number to code. If single number, will roughly split across all strata under stratified proportional to the strata sizes. If a vector of length strata, will code that many in each strata (ignoring treatment assignment unless by_tx=TRUE). For srs, will sample ignoring treatment assignment unless by_tx = TRUE
strata	Name of stratification to use. If null will do SRS.

`generate_population` *Generate a population with strata and outcomes and surrogate/predicted outcomes that depend on strata.*

Description

These strata are generated post-treatment, in that we generate n_strat strata within the treated and control units separately, and then generate outcomes. That said, DGP makes strata parameters the same for corresponding tx and co strata.

Usage

```
generate_population(
  N,
  n_strat,
  strata_config = c("balanced-exact", "balanced-approx", "unbalanced"),
  tau = 0,
  bias = c("none", "moderate", "large", "extreme-contrast"),
  residual_var = c("homogeneous", "heterogeneous", "extreme-contrast"),
```

```
  sd_y = 3,
  R2_target = 0.5,
  no_tx = FALSE,
  seed = NULL,
  oracle = TRUE
)
```

Arguments

no_tx	If TRUE, do not generate treatment assignment (Z) and set all outcomes to control potential outcome.

Details

TODO: Review the DGP – is what we want?

---

generate_sample_index *Given a full population, generate a vector of indicators for who is sampled.*

Description

Given a full population, generate a vector of indicators for who is sampled.

Usage

```
generate_sample_index(
  dat,
  n_code,
  Z = NULL,
  strata = NULL,
  p_tx = 0.5,
  seed = NULL
)
```

Arguments

n_code	Number to code. If single number, will roughly split across all strata under stratified proportional to the strata sizes. If a vector of length strata, will code that many in each strata (ignoring treatment assignment unless tx col passed). If tx col passed, then it is best to make sure the passed strata all are all treatment or all control, although it will interact strata with treatment indicator, and then order the strata as all control strata followed by all treatment strata. So if there is strata 1,2,3, you would get six strata of 1-0,2-0,3-0, and then 1-1,2-1,3-1. You would then need to pass a vector of length 6 for n_code (or a single number for total sample size). For SRS, will sample ignoring treatment assignment unless treatment column passed, and then either make two strata (a co and tx strata) and spread n_code across it, or take a vector of length 2 for n_code for control, then treatment.
Z	If NULL, sample ignoring treatment. If not null, sample within substrata defined by interacting strata with treatment.

strata	Name of stratification to use. If null will do SRS (i.e., treat all units as a single strata).
p_tx	Proportion of sample to allocate to treatment arm, if treatment arm is defined and n_code doesn't specify exact amounts for each strata. If NULL, then will allocation proportional to proportion actually treated in each strata.

`plot_strata_residuals` *Create visualization of residual structure by strata*

Description

Create visualization of residual structure by strata

Usage

```
plot_strata_residuals(dat, strata)
```

Arguments

dat	Data frame with Yobs, Yhat, Z, strata columns
strata	Stratification variable

Value

ggplot object

Index

calc_optimal_allocation_oracle, 2
calc_sampling_weights, 2
compare_stratifications, 3
compare_stratifications_precoding, 3
compute_variance_decomposition, 4
create_stratification_candidates, 4

est_ma_srs, 6
est_ma_srs_old, 6
est_ma_strat, 7
est_oracle, 8
est_subset, 8
evaluate_stratification, *3*, 9
evaluate_stratification_precoding, 10

generate_observed_sample, 11
generate_population, 11
generate_sample_index, 12

plot_strata_residuals, 13